Set in Python

29 July 2025 20:16

What is a set in Python?
A set is:

- An unordered mutable collection of unique elements.
- Similar to sets in mathematics.
- Defined using {} Or the set() constructor.

Key properties:

- **No duplicates:** Duplicate elements are automatically removed.
- **Unordered:** No, indexing elements are not stored in a specific order.
- **Mutable:** You can add or remove elements.
- **Heterogeneous:** Can store elements of different immutable types. **(int str, tuple, etc.)**

Creating Sets

Using {}

```
fruits = {'apple', 'banana', 'cherry', 'apple'}
print(fruits)
```

Output:

```
{'apple', 'cherry', 'banana'}
```

```
numbers = set([1,2,3,2,1])
print(numbers)
```

Output:

```
{1, 2, 3}
```

Creating an empty set

Important: {} Creates an empty dictionary, not a set. Use set () for an empty set.

```
empty_set = set()
print(type(empty_set))
```

Output:

```
<class 'set'>
```

Accessing Elements:
Sets are unordered, so no indexing is allowed.
You use loops:

```
colors = {"red", "green", "blue"}
for color in colors:
    print(color)
```

Output:

```
red
green
blue
```

Adding and Removing Elements:

```
animals = {"cat", "dog"}
animals.add("horse") # Add single element.
animals.update(["cow", "goat"]) # Add multiple elements.
print(animals)
```

Output:

```
{'cat', 'dog', 'goat', 'cow', 'horse'}
```

For removal:

```
animals = {"cat", "dog"}
animals.add("horse") # Add single element.
animals.update(["cow", "goat"]) # Add multiple elements.
print(animals)
animals.remove("cat")  # Remove items. Error if not found.
print(animals)
animals.remove("frog") # Error: KeyError
```

Output:

```
{'dog', 'horse', 'cow', 'goat', 'cat'}
{'dog', 'horse', 'cow', 'goat'}
Traceback (most recent call last):
  File "c:\Users\bluej\Documents\Students\Jiya
    animals.remove("frog") # Error:
    ^^^^^^^^^^^^^^^^^^^^^^
KeyError: 'frog'
```

```
animals = {"cat", "dog"}
animals.add("horse") # Add single element.
animals.update(["cow", "goat"]) # Add multiple elements.
print(animals)
animals.remove("cat")  # Remove items. Error if not found.
# animals.remove("frog") # Error:
animals.discard("frog")   # Remove item. No error if not found.
print(animals)
```

Output:

```
{'goat', 'horse', 'cat', 'cow', 'dog'}
{'goat', 'horse', 'cow', 'dog'}
```

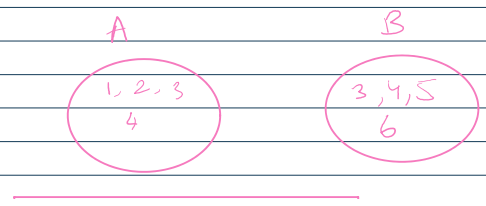
Common Set Operations
Python sets supports mathematical operations:

```
A = {1,2,3,4}
B = {3,4,5,6}

print('Union:', A | B)
print("Intersection: ", A & B)
print("Difference(A - B):", A - B)
print("Symmetric Difference:", A^B)
```

Union ∪

intersection ∩

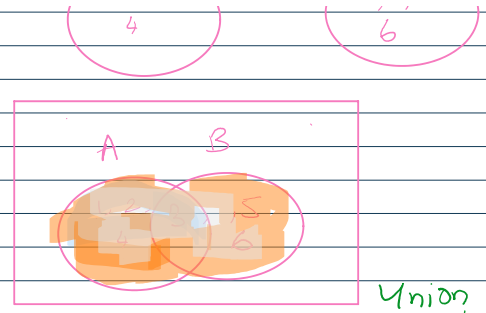


```
print("Union: ", A | B)
print("Intersection: ", A & B)
print("Difference(A - B):", A - B)
print("Symmetric Difference:", A^B)
```

Output:

```
Union: {1, 2, 3, 4, 5, 6}
Intersection:  {3, 4}
Difference(A - B): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
```

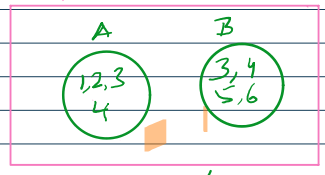
we discard the common elements from 1st set, and we discard all the elements from 2nd set

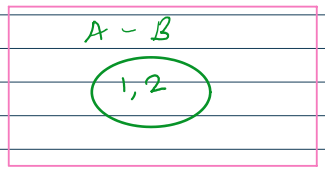


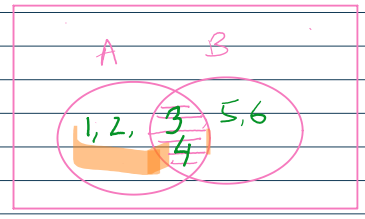
A U B

Mathematical symbol

Union (all distinct elements from both sets)







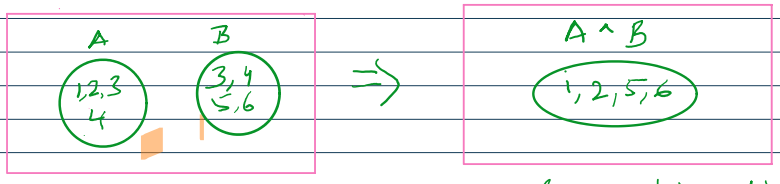
A ∩ B

Mathematical symbol

Intersection: (all common elements from both sets).

```
Union: {1, 2, 3, 4, 5, 6}
Intersection:  {3, 4}
Difference(A - B): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
```

here we discard the common elements from both the set.



Symmetric difference

Membership Testing

```
fruits = {'apple', 'banana', 'mango', 'grapes'}
print('apple' in fruits)  #True
print("grapes" not in fruits) #False
print("cherry" not in fruits) #True
```

Output:

```
True
False
True
```

Frozen Set(Immutable Set)
A frozen set is an immutable version of a set.
You can use it as a dictionary key or element of another set.

```
fs = frozenset([1, 2, 3])
print(fs)
print(type(fs))
```

Output:

```
frozenset({1, 2, 3})
<class 'frozenset'>
```

```
fs = frozenset([1, 2, 3])
print(fs)
print(type(fs))
fs.add(4)
```

Output:

```
frozenset({1, 2, 3})
<class 'frozenset'>
Traceback (most recent call last):
  File "c:\Users\bluej\Documents\Students\JiyaGaurDeensAca
    fs.add(4)
    ^^^^^^
AttributeError: 'frozenset' object has no attribute 'add'
```

Homework
Here are **10 practice questions** on Python sets (ranging from beginner to advanced) and then a **comparison table of list, tuple, set, and dict**.

---

10 Practice Questions on Python Sets

Q1. Unique Words from a Sentence

Write a Python program that takes a sentence as input and prints all the **unique words** in it using a set.

Q2. Remove Duplicates from a List

Given a list of integers with duplicates, write a program to **remove duplicates** and display the unique elements.

Q3. Common Elements in Two Sets

Create two sets of student names (class A and class B).
Write a program to:

* Print students who are present in both classes.
* Print students who are only in class A but not in class B.

Q4. Set Operations (Union, Intersection, etc.)

Given:

```python
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Write a program to compute and print:

* Union
* Intersection
* Difference (A - B)
* Symmetric difference

Q5. Frequency of Characters

Take a string and print the **set of unique characters** present in it.

Q6. Set Update Operations

Create a set of favorite movies.

* Use `.add()` to add a new movie.
* Use `.update()` to add multiple movies.
* Use `.remove()` and `.discard()` and demonstrate the difference.

Q7. Subset and Superset

Write a program to check if:

* `{1, 2}` is a subset of `{1, 2, 3, 4}`
* `{1, 2, 3, 4}` is a superset of `{2, 3}`

Q8. Immutable Set

Create a `frozenset` from a list of numbers and try to modify it.
Observe and explain what happens.

Q9. Remove All Items from a Set

Create a set, display it, then remove all elements using `.clear()` and print the set.

Q10. Real-Life Scenario: Unique Visitors

A website stores IP addresses of visitors in a list (with duplicates).
Write a Python program to convert it into a set and display the **count of unique visitors**.

Comparison: List vs Tuple vs Set vs Dict in Python

| Feature | **List** | **Tuple** | **Set** |

Dict
Syntax	`[ ]`	`( )`	`{ }` or `set()`	`{key: value}`
Order	Ordered	Ordered	Unordered	Ordered (Python 3.7+)
Mutable?	Mutable	Immutable	Mutable	Mutable
Duplicates?	Allowed	Allowed	**Not allowed**	Keys not allowed, values allowed
Indexing?	Yes (0, 1, 2...)	Yes	**No indexing**	Keys, not indexes
Use cases	General-purpose storage	Fixed data (constants)	Membership tests, set ops	Key-value mappings

---

Key Takeaways

* **List** – ordered, mutable, duplicates allowed.
* **Tuple** – ordered, immutable, duplicates allowed.
* **Set** – unordered, unique values, no indexing.
* **Dict** – key-value mapping, keys unique.